01 Install Flutter and Dart SDK & Write a simple Dart program to understand the language basics

main.dart

```dart
void main() {
    int a = 10;
    int b = 5;
    double add = a + b;
    double sub = a - b;
    double mul = a * b;
    double div = a / b;
    print('Addition: $add');
    print('Subtraction: $sub');
    print('Multiplication: $mul');
    print('Division: $div');
}
```

Output (Console)

```
Addition: 15
Subtraction: 5
Multiplication: 50
Division: 2.0
```



Flutter SDK



Dart SDK

02

Explore various Flutter widgets and Implement layout structures using Row, Column and Stack



Text
(Display)

Container
(Display panel)

Row
(Horizontal buttons)

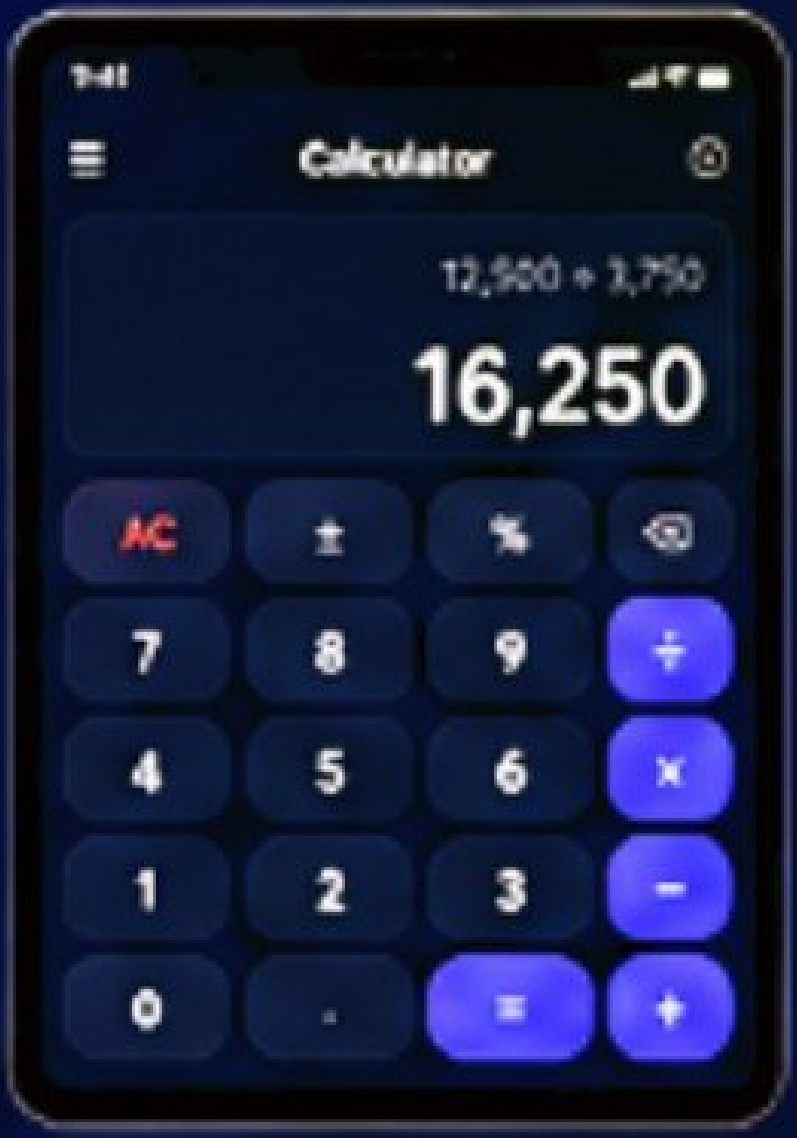
Column
(Vertical layout)

Stack
(Background & glow effects)

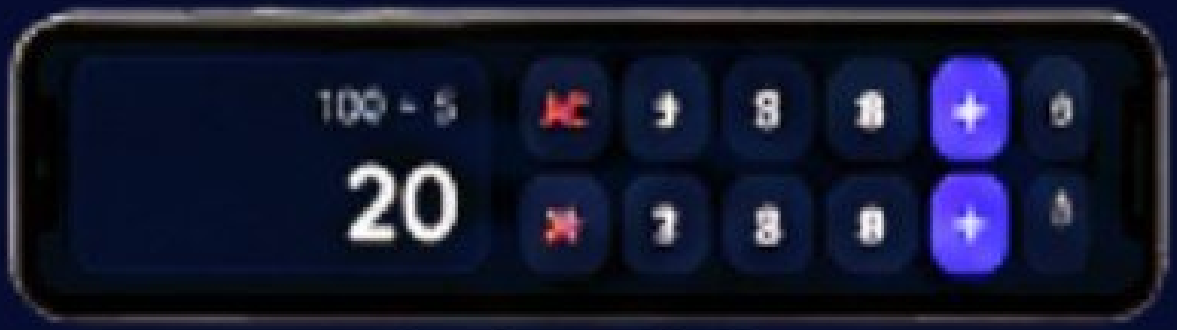
03 Design a responsive UI that adapts to different screen sizes using media queries and breakpoints



Mobile
(360 × 800)



Tablet
(768 × 1024)



Landscape Mode

04 Setign a navigative between different screens using Navigator and Implement navigation with named routes



Standard (360 × 1020)

Scientific (768 × 1024)

History (Screen)

Named Routes (main.dart)

```
routes: {
  "/": (context) => CalculatorScreen(),
  "/scientific": (context) => ScientificScreen(),
  "/history": (context) => HistoryScreen(),
},
```

05 Learn about stateful and stateless widgets and implement state management using setState and Provider

Using setState (Calculator display)

```
setState(() {
  _currentInput = _currentInput + value;
  _result = _calculate();
});
```

Using Provider (History management)

```
class HistoryProvider with
  ChangeNotifier {
  final List<String> _history = [];

  void addHistory(String result) {
    _history.add(result);
    notifyListeners();
  }
}
```



StatefulWidget
(Calculator Screen)



StatelessWidget
(Reusable Widgets)



Provider
(State Management)

06

Create custom widgets for specific UI elements and Apply styling using themes and custom styles

Custom CalculatorButton




```
class CalculatorButton
  extends StatelessWidget {
  final String text;
  final VoidCallback onTap;
  ...
}
```

Custom DisplayPanel

25 × (10 + 5)

375

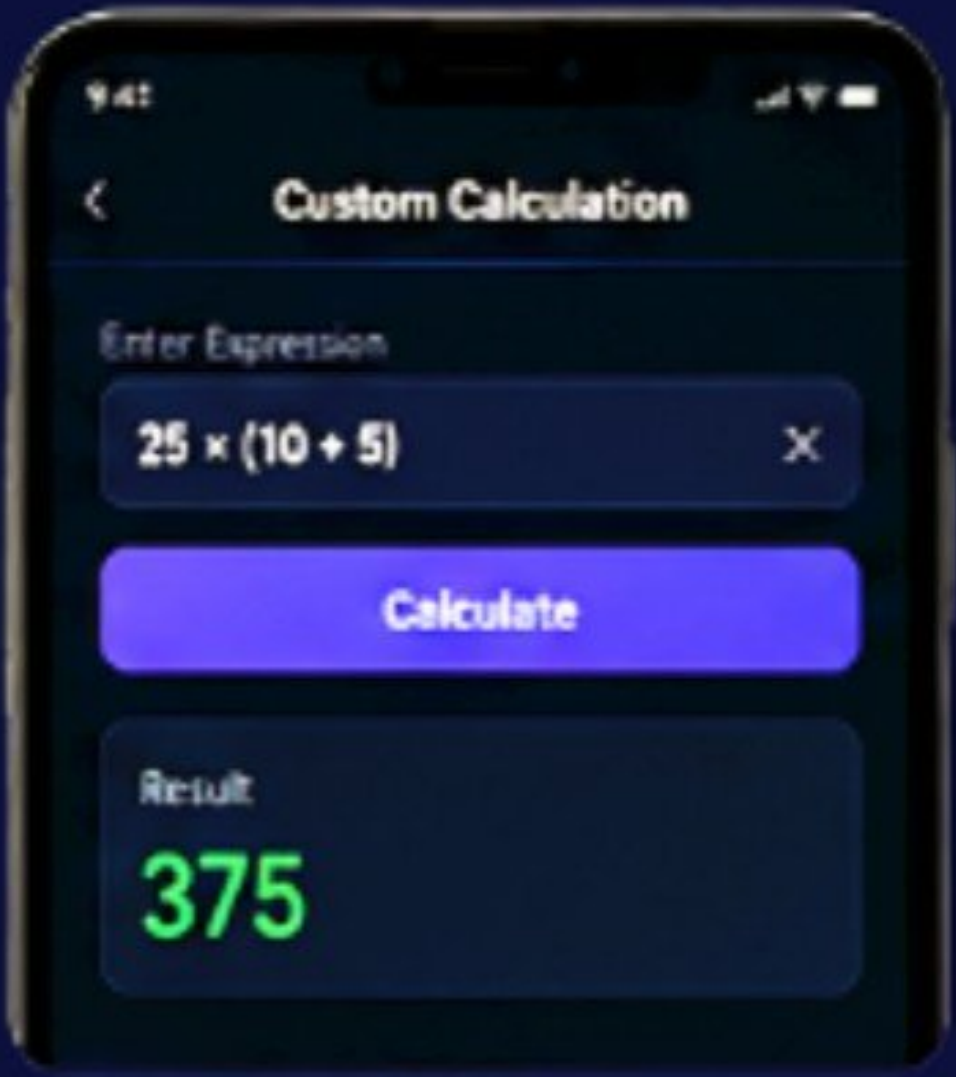
```
class DisplayPanel
  extends StatelessWidget {
  final String expression;
  final String value;
  ...
}
```

Theme and Custom Styles

```
final ThemeData appTheme = ThemeData(
  brightness: Brightness.dark,
  primaryColor: Color(0xFF6CA3FF),
  scaffoldBackgroundColor: Color(0xFF0D1A),
  fontFamily: 'Poppins',
);
```



07 Design a form with various input fields and implement form validation and error handling



Validation Examples

- Empty expression
  Please enter a expression
- Invalid expression
  Invalid mathematical expression
- Division by zero
  Cannot divide by zero

08

Add animations to UI elements using Flutter's animation framework and Experiment with different types of animations

7

Button Press Animation
(Scale Animation)

100

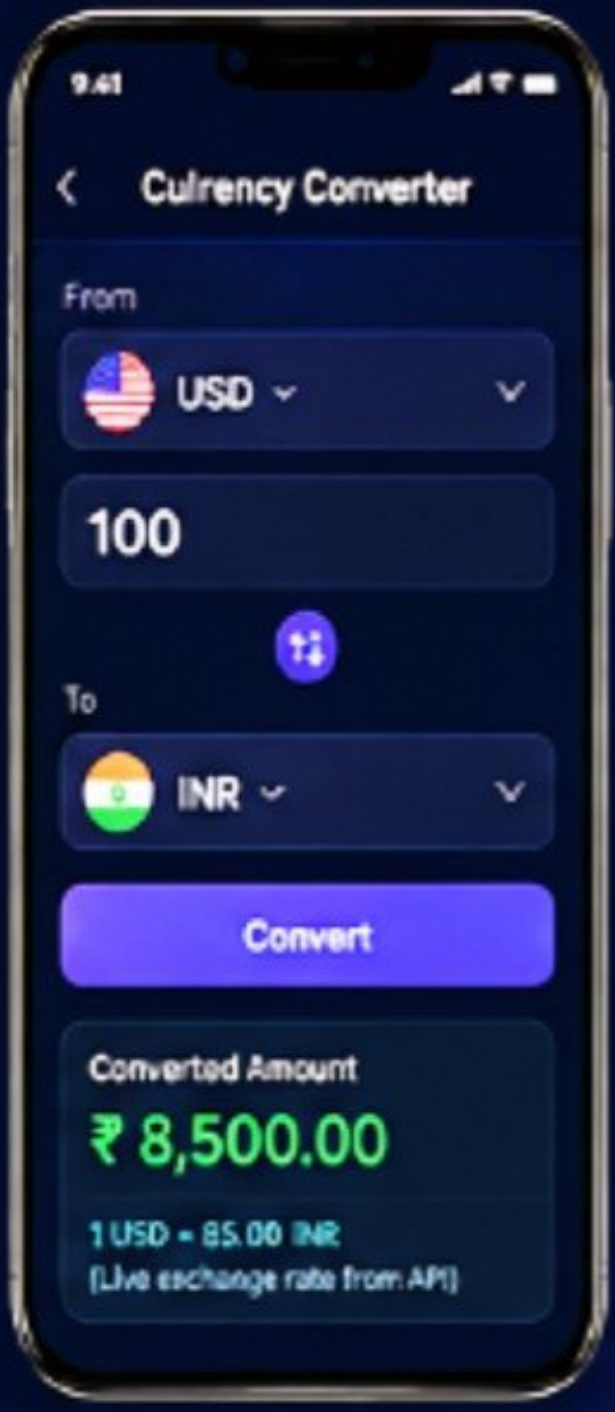
200

Fade Animation
(Display Change)



Slide Animation (History Panel)

09 Fetch data from a REST API and Display the fetched data in a meaningful way in the UI



REST API Integration
(currency_api_service.dart)

```dart
final url =
'https://api.exchangerate-api.com/v4/
 latest/USD';

final response = await
http.get(Uri.parse(url));

if (response.statusCode == 200) {
 final data = jsonDecode(response.body);
 final rate = data['rates']['INR'];
 return rate;
}
```

API Response (JSON)

```json
{
  "base": "USD",
  "rates": {
    "INR": 85.00,
    "EUR": 0.92,
    "GBP": 0.78
  }
}
```

10 Write unit tests for UI components and Use Flutter's debugging tools to identify and fix issues

Unit Test (calculator_logic_test.dart)

```dart
import 'package:flutter_test/flutter_test.dart';
import 'package:calculator_app/logic/calculator_logic.dart';

void main() {
  test('Addition test', () {
    expect(CalculatorLogic.add(10, 20), 30);
  });
  test('Multiplication test', () {
    expect(CalculatorLogic.multiply(5, 4), 20);
  });
  test('Division test', () {
    expect(CalculatorLogic.divide(20, 5), 4);
  });
```

Test Results

```
✓ Addition test
✓ Multiplication test
✓ Division test

All tests passed!
3 tests, 0 failures
```

Flutter Debugging Tools

